

Types of AI Agent Architectures

- Five basic types in order of increasing generality/ Functionality and also complexity of the system:
 - Simple reflex agents
 - Reflex agents with state/model (Model-based Agents)
 - Goal-based agents
 - Utility-based agents
 - Learning Agents

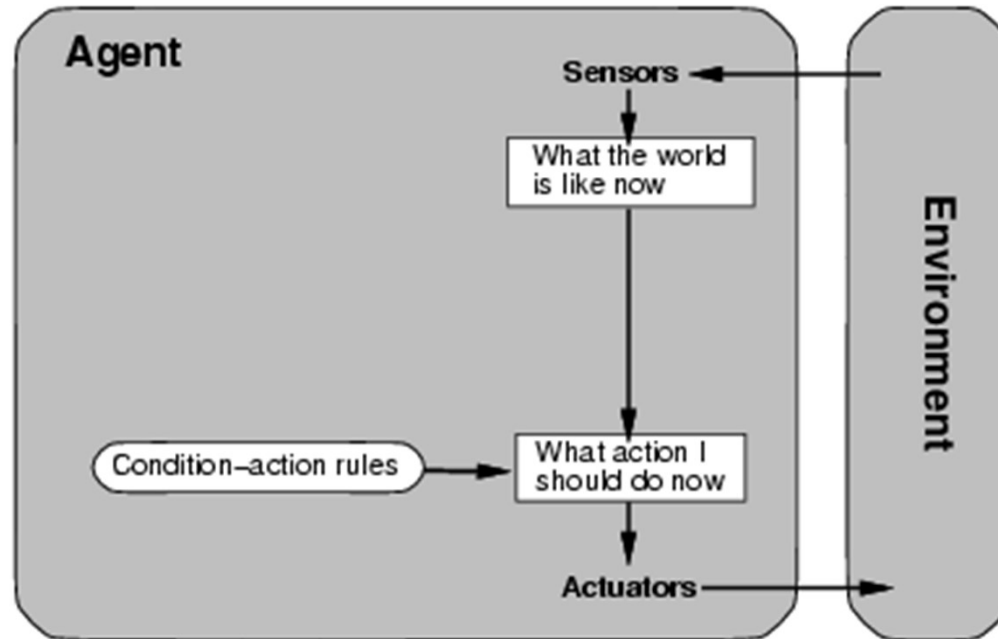
Simple reflex agents

Architecture: These agents operate based on condition-action rules. They perceive the current state of the environment and choose an action based on that state alone, without considering history or potential future states.

Key components:

- Sensors: To perceive the environment
- Condition-action rules: If (condition) then (action)
- Actuators: To perform actions

Simple reflex agents



Vacuum cleaner AI agent

Simple Reflex Agent

Functionality:

- React to immediate sensor inputs without any memory or internal state.
- Use a set of condition-action rules to decide whether to clean, stop, or change direction.

Implementation Steps:

- **Sensors**: Basic dirt detection sensors to detect dirt or debris.
- **Actuators**: Basic movement control (forward, backward, turn left, turn right).
- **Rules**: If dirt detected, clean; if no dirt, move forward; if obstacle detected, turn right.

Example Rule:

- If (dirt_detected) then (clean).
- If (obstacle_detected) then (turn_right).

Vacuum cleaner AI agent

```
function REFLEX-VACUUM-AGENT( [location,status] ) returns an action
  if status = Dirty then return Suck
  else if location = A then return Right
  else if location = B then return Left
```

Model-Based Reflex Agents

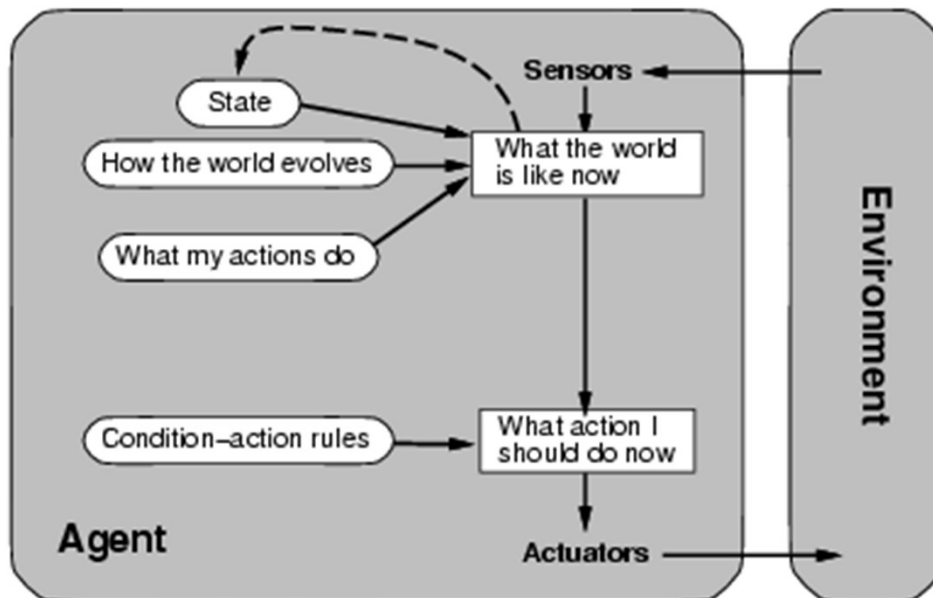
- Recall the **agent function** that maps from percept histories to actions:

$$[f: \mathcal{P}^* \rightarrow \mathcal{A}]$$

- An agent program can implement an agent function by maintaining an **internal state**.
- The internal state can contain information about the state of the external environment.
- The state depends on the history of percepts and on the history of actions taken:

$$[f: \mathcal{P}^*, \mathcal{A}^* \rightarrow \mathcal{S} \rightarrow \mathcal{A}] \text{ where } \mathcal{S} \text{ is the set of states.}$$

Model-based reflex agents



- Know how world evolves
 - Overtaking car gets closer from behind
- How agents actions affect the world
 - Wheel turned clockwise takes you right
- Model base agents update their state

```
function REFLEX-AGENT-WITH-STATE(percept) returns action
  static: state, a description of the current world state
         rules, a set of condition-action rules

  state ← UPDATE-STATE(state, percept)
  rule ← RULE-MATCH(state, rules)
  action ← RULE-ACTION[rule]
  state ← UPDATE-STATE(state, action)
  return action
```

Vacuum cleaner AI agent

Model-Based Reflex Agent

Functionality:

- Maintain an internal state to remember parts of the environment.
- Use a map or a model to navigate more efficiently.

Implementation Steps:

- **Sensors**: Same as above, plus sensors for mapping (e.g., infrared, ultrasonic).
- **Internal State**: Maintain a map of the area to track cleaned and uncleaned regions.
- **Rules**: Use the map to decide the next move, avoid redundant cleaning, and optimize path planning.

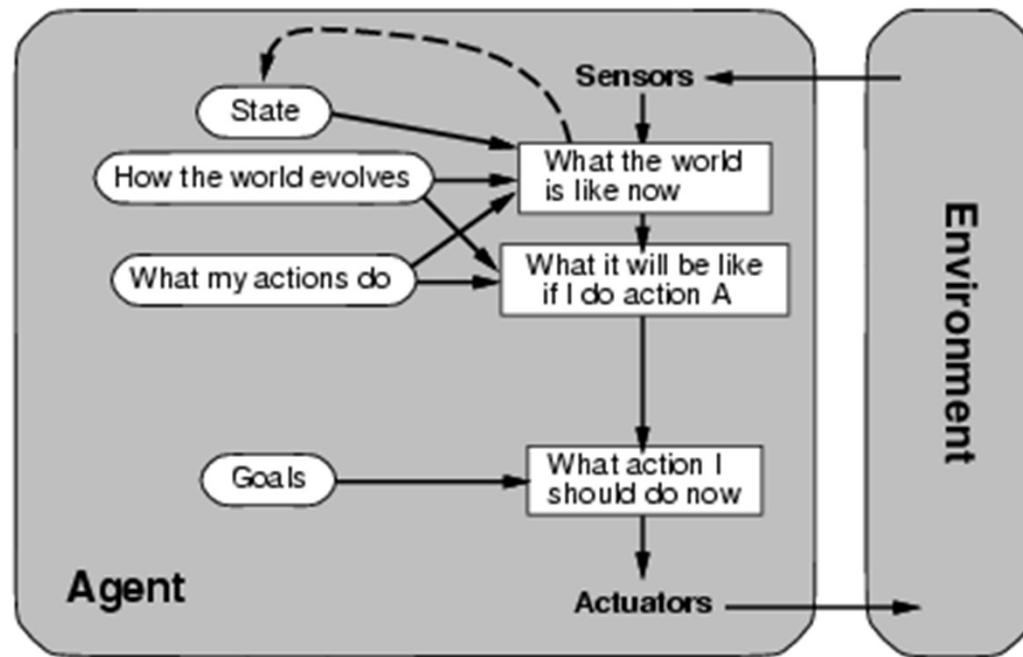
Example Rule:

- If (dirt_detected and location_uncleaned) then (clean).
- If (location_cleaned) then (update_map and move_to_uncleaned_area).

Goal-based agents

- knowing state and environment? Enough?
 - Taxi can go left, right, straight
- Have a goal
 - A destination to get to
- Uses knowledge about a goal to guide its actions
 - E.g., Search, planning

Goal-based agents



- Reflex agent breaks when it sees brake lights. Goal based agent reasons
 - Brake light -> car in front is stopping -> I should stop -> I should use brake

Vacuum cleaner AI agent

Goal-Based Agent

Functionality:

- Set and pursue specific goals, such as fully cleaning a room.
- Plan actions to achieve these goals.

Implementation Steps:

- Goals: Define goals like "clean all dirty areas" or "return to docking station."
- Path Planning: Implement algorithms like A* for pathfinding to navigate to goals.
- Decision Making: Prioritize tasks (e.g., cleaning vs. returning to dock).

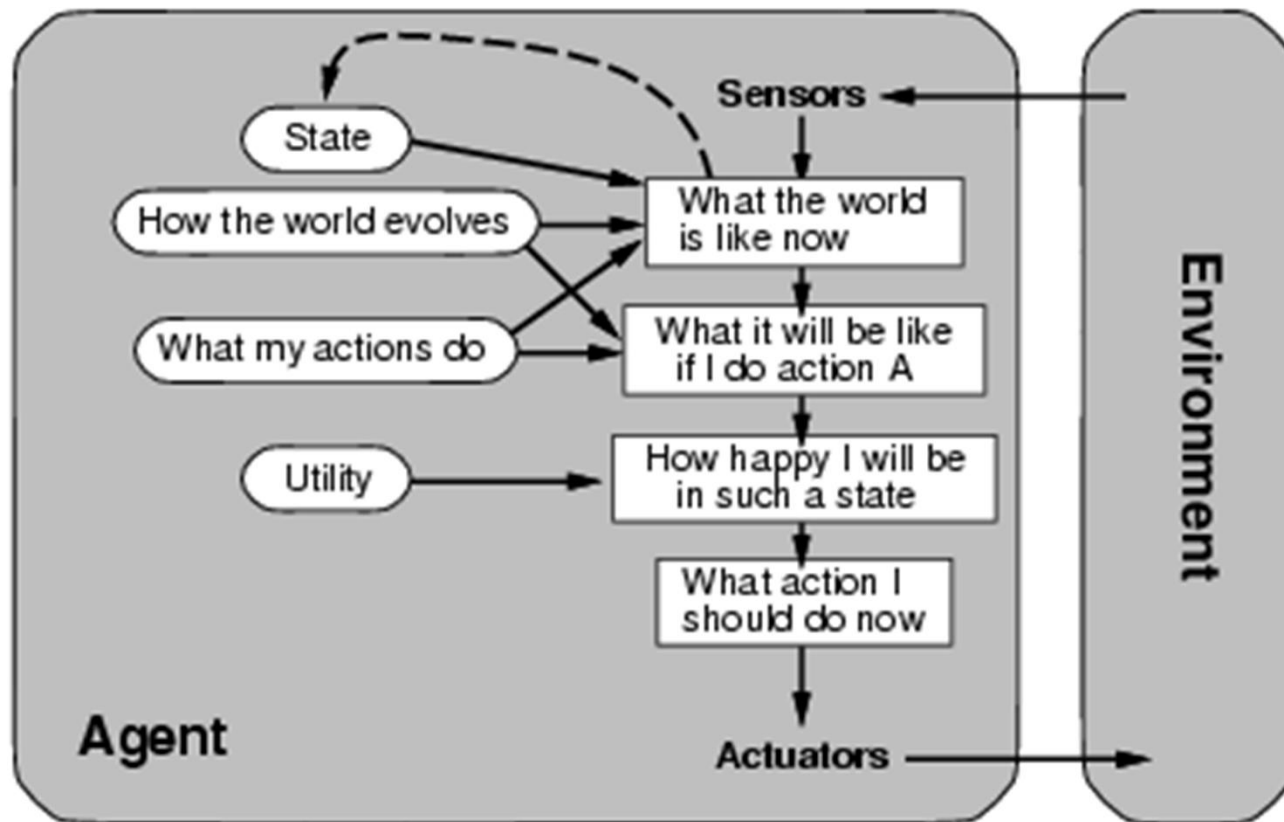
Example Decision Process:

- Set goal: "clean all detected dirt."
- Plan path to the nearest dirty spot using the map.
- Execute cleaning until the goal is achieved.

Utility-based agents

- Goals are not always enough
 - Many action sequences get taxi to destination
 - Consider other things. How fast, how safe.....
- A utility function maps a state onto a real number which describes the associated degree of “happiness”, “goodness”, “success”.

Utility-based agents



Vacuum cleaner AI agent

Utility-Based Agent

Functionality:

- Optimize cleaning based on a utility function, such as energy efficiency, time efficiency, or thoroughness.
- Evaluate different actions and choose the one with the highest utility.

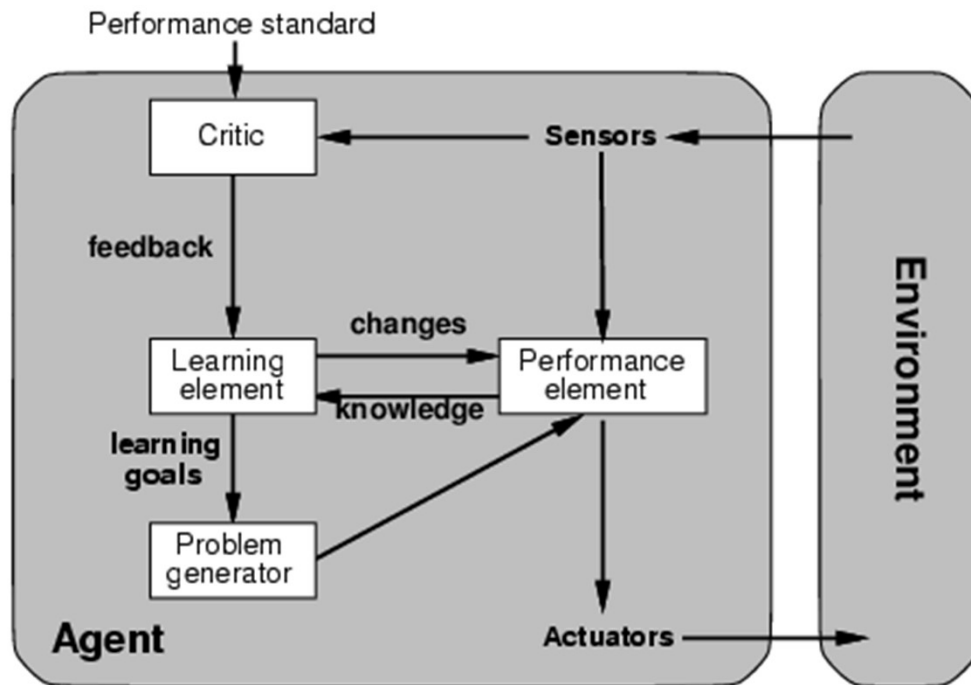
Implementation Steps:

- Utility Function: Define a function that evaluates outcomes (e.g., cleaning quality, battery usage).
- Decision Making: Choose actions that maximize utility (e.g., prioritize high dirt areas or low energy paths).
- Balancing Trade-offs: Consider factors like cleaning efficiency vs. energy consumption.

Example Utility Calculation:

- $Utility = (cleaning_quality) - (energy_cost)$.
- If (cleaning_quality high) and (energy_cost low) then (choose_action).

Learning agents



- Performance element is what was previously the whole agent
 - Input sensor
 - Output action
- Learning element
 - Modifies performance element.

Vacuum cleaner AI agent

Learning Agent

Functionality:

- - Learn from experience and adapt to new environments or changes.
- - Improve performance over time by adjusting strategies.

Implementation Steps:

- Learning Mechanisms: Implement supervised, unsupervised, or reinforcement learning algorithms.
- Feedback Loop: Use data from past cleaning sessions to improve strategies (e.g., adjust cleaning patterns based on the type of dirt detected in different rooms).
- Exploration and Exploitation: Balance exploring new strategies and exploiting known effective strategies.

Example Learning Process:

- Reinforcement Learning: Use rewards for successful cleaning (e.g., positive reward for cleaning dirt, negative reward for hitting obstacles).
- Supervised Learning: Train on labeled data to recognize different types of dirt and optimize cleaning strategies accordingly.

Summary of Evolution

1. **Simple Reflex Agent:** Reacts to immediate conditions (basic cleaning).
2. **Model-Based Reflex Agent:** Uses a map to avoid redundant cleaning.
3. **Goal-Based Agent:** Sets and achieves specific cleaning goals.
4. **Utility-Based Agent:** Optimizes actions based on a utility function.
5. **Learning Agent:** Learns and adapts to improve cleaning efficiency over time.